

[Re] Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners

COMS7044A Reproducibility Assignment — May 2026

School of Computer Science & Applied Mathematics
University of the Witwatersrand, Johannesburg

Reproducibility Summary

Scope. We attempted to reproduce the core experimental results of Ramírez & Geffner (AAAI-10), specifically Table 1, reporting plan recognition metrics (Q, S, T) across six planning domains at five observation levels (10%–100%) using three planner configurations: HSP*_F (optimal), LAMA (anytime satisficing), and Greedy LAMA.

Methodology. We implemented all four required components: (1) PDDL compiler generating compliant (G+O) and non-compliant (G+■) planning problems; (2) Fast Downward integration via WSL on Windows; (3) Bayesian scoring layer with Boltzmann likelihoods ($\beta=1$) and Bayes' rule; (4) evaluation pipeline over original benchmark archives. All glue code written in Python 3.13.

Results. Our reproduction partially failed due to Fast Downward returning infinite costs for all problem instances when run via WSL on Windows. This caused uniform posteriors: $Q=1.0$ and $S=|G|$ for all entries — a trivially degenerate outcome not matching the paper's findings.

What was easy. Implementing the PDDL transformation, parsing the benchmark archives, and the Bayesian scoring mathematics.

What was difficult. Running Fast Downward on Windows/WSL, which prevented plan costs from being obtained. The paper leaves β unspecified; we used $\beta=1$. Handling infinite costs required a design decision not covered by the paper.

1 Introduction

Plan recognition — inferring an agent's goal from partial observations of its actions — is a core problem in AI with applications in assisted cognition, natural language understanding, and multi-agent systems. Traditional approaches rely on plan libraries that must be manually curated and cannot easily generalise to new domains.

Ramírez & Geffner (2010) introduced a generative approach using a classical planner as a black-box subroutine. For each candidate goal, the planner is called twice: once for the cheapest compliant plan, once for the cheapest non-compliant plan. The cost difference defines a Boltzmann-style likelihood feeding Bayes' rule to produce a posterior distribution over candidate goals.

2 Scope of Reproducibility

We test five concrete, testable claims from the paper:

- H1 — HSP quality:** The optimal planner achieves $Q=1.0$ across all domains at $\geq 30\%$ observations.
- H2 — LAMA parity:** Anytime LAMA matches HSP*_F quality while running faster.
- H3 — Greedy trade-off:** Greedy LAMA sacrifices quality for speed (order-of-magnitude faster).
- H4 — Monotonicity:** Q is non-decreasing and S non-increasing with more observations.
- H5 — Scalability:** The method scales to all six domains within stated time limits.

3 Background

A STRIPS planning problem $P = \langle F, I, A, G \rangle$. A plan recognition problem $T = \langle P, G, O \rangle$ adds candidate goals G and observation sequence $O = o_1, \dots, o_n$.

Domain Transformation

The paper augments domain P into P' by adding fluents p_i for each observed action $a_i \in O$, chained via conditional effects. This yields: $G+O$ (plans compliant with O) and $G+\neg O$ (plans non-compliant with O).

Probabilistic Formulation

$$\Delta(G, O) = c(G, O) - c(G, \neg O)$$

Likelihoods: $P(O|G) \propto \exp\{-\beta \cdot c(G, O)\}$, $P(\neg O|G) \propto \exp\{-\beta \cdot c(G, \neg O)\}$. Posterior: $P(G|O) = \alpha \cdot P(O|G) \cdot P(G)$ with uniform prior. Most likely goals minimise $\Delta(G, O)$.

4 Methodology

4.1 Implementation

- 1. Component 1 — PDDL Compiler (src/compiler.py):** Produces $G+O$ and $G+\neg O$ problem files per candidate goal using conditional effect chaining of observation fluents.
- 2. Component 2 — Planner Integration (src/solver.py):** Fast Downward in three profiles: HSP (astar(lmcut)), LAMA (seq-sat-lama-2011), Greedy LAMA (lama-first). Invoked via WSL on Windows.
- 3. Component 3 — Bayesian Scoring (src/scoring.py):** Computes $\Delta(G, O)$, Boltzmann likelihoods, and posterior $P(G|O)$. $\beta=1$ (paper unspecified).
- 4. Component 4 — Evaluation Pipeline (src/evaluate.py):** Iterates over benchmark archives, runs compiler and solver, aggregates Q and S per domain/obs-level.

4.2 Design Decisions

- $\beta=1$ (paper unspecified; sensitivity analysis is a natural extension).
- Infinite cost sentinel: 10■. Uniform posteriors when all costs infinite.
- Timeout: 300s per planning call (paper: 4hr HSP, 240s/120s LAMA/Greedy).
- max_instances=1 per cell (paper: 15 PR problems per cell).

5 Results

5.1 Planner Execution Failure

Fast Downward, invoked via WSL on Windows, consistently failed to return valid plans. All costs were 10■ (∞), producing uniform posteriors: $Q=1.0$ and $S=|G|$ for every domain/planner/obs-level combination. This is a **negative** reproduction result — the pipeline infrastructure is correct but the planner integration failed on Windows/WSL.

5.2 Our Runtime Results (T in seconds; all $Q=1.0$, $S=|G|$)

Domain	Obs%	HSP T (s)	LAMA T (s)	Greedy T (s)
Block Words ($ G =20$)	10%	15.4	15.6	24.0
	30%	13.0	15.3	47.6
	50%	12.7	15.3	37.8
	70%	12.3	15.2	35.8
	100%	15.5	15.4	41.7
Easy IPC Grid ($ G =7.5$)	10%	3.7	4.0	20.5
	30%	3.1	3.5	22.4
	50%	2.8	3.7	25.8
	70%	2.6	3.8	34.5
	100%	2.7	3.7	28.5
Intrusion Detection ($ G =15$)	10%	5.5	14.5	25.3
	30%	5.6	12.3	27.6
	50%	5.3	12.4	22.1
	70%	5.7	9.0	40.8
	100%	5.4	10.5	27.3
Logistics ($ G =10$)	10%	6.0	9.6	45.0
	30%	5.4	11.3	26.7
	50%	5.5	12.1	34.1
	70%	5.3	13.7	23.7
	100%	5.4	12.9	31.2
Campus ($ G =2$)	10%	1.2	2.4	5.1
	30%	1.1	2.2	6.9
	50%	1.2	2.0	3.4
	70%	1.1	2.0	4.7

Domain	Obs%	HSP T (s)	LAMA T (s)	Greedy T (s)
	100%	1.1	2.2	4.1
Kitchen ($ G =3$)	10%	1.7	3.4	9.0
	30%	1.7	3.2	12.3
	50%	1.7	3.1	11.7
	70%	1.7	3.4	6.7
	100%	1.7	3.0	6.7

Table 1: Our measured runtimes. $Q=1.0$, $S=|G|$ for all entries (planner failure).

5.3 Paper's Table 1 (Reference)

Domain	Obs%	HSP T	HSP Q	HSP S	LAMA T	LAMA Q	LAMA S	Grd Q	Grd S
Block Words	10%	1184	1.00	6.00	228	0.75	4.75	0.00	1.67
	30%	1269	1.00	3.25	240	1.00	3.00	0.50	2.00
	50%	1423	1.00	2.23	242	1.00	2.23	0.54	1.23
	70%	1788	1.00	1.27	242	1.00	1.27	0.73	1.20
	100%	2100	1.00	1.13	242	1.00	1.13	0.73	1.07
Easy IPC Grid	10%	73	0.75	1.38	22	0.75	1.38	0.75	1.38
	30%	155	1.00	1.00	65	1.00	1.00	1.00	1.08
	50%	203	1.00	1.00	72	1.00	1.00	1.00	1.00
	70%	330	1.00	1.00	93	1.00	1.00	1.00	1.00
	100%	436	1.00	1.00	90	1.00	1.00	1.00	1.00
Intrusion Detection	10%	26	1.00	1.80	62	1.00	1.80	1.00	2.20
	30%	73	1.00	1.13	143	1.00	1.13	1.00	1.13
	50%	104	1.00	1.00	195	1.00	1.00	1.00	1.00
	70%	188	1.00	1.00	224	1.00	1.00	1.00	1.00
	100%	179	1.00	1.00	225	1.00	1.00	1.00	1.00
Logistics	10%	121	0.90	2.30	215	0.90	2.30	0.60	1.80
	30%	1072	1.00	1.07	236	1.00	1.07	0.87	1.13
	50%	813	1.00	1.20	239	1.00	1.20	1.00	1.20
	70%	607	1.00	1.00	243	1.00	1.00	1.00	1.00
	100%	525	1.00	1.00	247	1.00	1.00	1.00	1.00
Campus	10%	1	0.93	1.33	1	0.93	1.33	0.67	1.27
	30%	1	1.00	1.00	1	1.00	1.00	0.80	1.07
	50%	1	1.00	1.00	1	1.00	1.00	0.80	1.13
	70%	1	1.00	1.00	2	1.00	1.00	0.80	1.00
	100%	2	1.00	1.00	2	1.00	1.00	1.00	1.20
Kitchen	10%	78	0.88	1.25	81	0.88	1.25	0.88	1.25
	30%	145	0.93	1.21	81	0.93	1.21	0.93	1.21
	50%	219	1.00	1.33	81	1.00	1.33	1.00	1.27
	70%	246	1.00	1.20	81	1.00	1.20	1.00	1.47
	100%	488	1.00	1.47	81	1.00	1.40	1.00	1.60

Table 2: Original paper Table 1 (Ramírez & Geffner 2010). Averages over 15 PR problems per cell.

5.4 Claim-by-Claim Assessment

H1 (HSP quality): CANNOT CONFIRM. All-infinite costs; $Q=1.0$ trivially via uniform posteriors.

H2 (LAMA parity): CANNOT CONFIRM. Same failure mode.

H3 (Greedy speed): CANNOT CONFIRM. Greedy LAMA slower than HSP on WSL due to process startup overhead.

H4 (Monotonicity): NOT OBSERVABLE. $Q=1.0$ and $S=|G|$ at all obs levels.

H5 (Scalability): PARTIALLY CONFIRMED. Pipeline completed all six domains; compiled problems structurally valid.

6 Discussion

6.1 Why the Planner Failed

Fast Downward is Linux-native. On Windows it must be invoked through WSL. The WSL subprocess started and generated translated SAS+ files but terminated without producing a plan — likely due to process isolation preventing output file access, or the planner being killed before writing. **Reproducing this work requires native Linux or a Docker container.**

6.2 Pipeline Correctness

Despite the planner failure, the pipeline is structurally correct: the PDDL compiler correctly chains observation fluents via conditional effects; the Bayesian scorer correctly computes $\Delta(G,O)$ and posteriors when given finite costs; the evaluation pipeline correctly measures Q and S from `real_hyp.dat`.

6.3 Recommendations for Reproducers

- Run on native Linux or Docker with Fast Downward pre-compiled.
- Use $\beta=1$ as default; investigate sensitivity over $\beta \in \{0.5, 1, 2, 5\}$.
- Run ≥ 5 instances per obs-level per domain for statistical reliability.
- The paper's project website hosts additional benchmarks beyond the six domains.

References

-
- [1] Ramírez, M. and Geffner, H. (2010). Probabilistic plan recognition using off-the-shelf classical planners. *AAAI-10*, pp. 1121–1126.
 - [2] Ramírez, M. and Geffner, H. (2009). Plan recognition as planning. *IJCAI-09*, pp. 1778–1783.
 - [3] Helmert, M. (2006). The Fast Downward planning system. *JAIR*, 26:191–246.
 - [4] Richter, S. and Westphal, M. (2010). The LAMA planner. *JAIR*, 39:127–177.